

7 DAYS

Upcoming Tech Talk: Build an API With Strapi



Community



TUTORIAL

How To Set Up Django with Postgres, Nginx, and Gunicorn on Ubuntu 20.04

Nginx

Python

PostgreSQL

Django

Python Frameworks

Databases

Ubuntu 20.04

By [Erin Glass](#)

Published on June 29, 2020 62.9k

English

Not using Ubuntu 20.04?

Choose a different version or distribution.

Ubuntu 20.04

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

[I understand](#)

code locally, but for anything even slightly production related, a more secure and powerful web server is required.

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

serve our apps.

Prerequisites and Goals

In order to complete this guide, you should have a fresh Ubuntu 20.04 server instance with a basic firewall and a non-root user with `sudo` privileges configured. You can learn how to set this up by running through our [initial server setup guide](#).

We will be installing Django within a virtual environment. Installing Django into an environment specific to your project will allow your projects and their requirements to be handled separately.

Once we have our database and application up and running, we will install and configure the Gunicorn application server. This will serve as an interface to our application, translating client requests from HTTP to Python calls that our application can process. We will then set up Nginx in front of Gunicorn to take advantage of its high performance connection handling mechanisms and its easy-to-implement security features.

Let's get started.

Installing the Packages from the Ubuntu Repositories

To begin the process, we'll download and install all of the items we need from the Ubuntu repositories. We will use the Python package manager `pip` to install additional components a bit later.

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

```
$ sudo apt update
$ sudo apt install python3-pip python3-dev libpq-dev postgresql postgresql-contrib ngi
```

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

```
$ sudo apt update
$ sudo apt install python-pip python-dev libpq-dev postgresql postgresql-contrib nginx
```

This will install `pip`, the Python development files needed to build Gunicorn later, the Postgres database system and the libraries needed to interact with it, and the Nginx web server.

Creating the PostgreSQL Database and User

We're going to jump right in and create a database and database user for our Django application.

By default, Postgres uses an authentication scheme called "peer authentication" for local connections. Basically, this means that if the user's operating system username matches a valid Postgres username, that user can login with no further authentication.

During the Postgres installation, an operating system user named `postgres` was created to correspond to the `postgres` PostgreSQL administrative user. We need to use this user to perform administrative tasks. We can use `sudo` and pass in the username with the `-u` option.

Log into an interactive Postgres session by typing:

```
$ sudo -u postgres psql
```

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

Note: Every Postgres statement must end with a semi-colon, so make sure that your command ends with one if you are experiencing issues.

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

Afterwards, we'll modify a few of the connection parameters for the user we just created. This will speed up database operations so that the correct values do not have to be queried and set each time a connection is established.

We are setting the default encoding to `UTF-8`, which Django expects. We are also setting the default transaction isolation scheme to "read committed", which blocks reads from uncommitted transactions. Lastly, we are setting the timezone. By default, our Django projects will be set to use `UTC`. These are all recommendations from the Django project itself:

```
postgres=# ALTER ROLE myprojectuser SET client_encoding TO 'utf8';
postgres=# ALTER ROLE myprojectuser SET default_transaction_isolation TO 'read committed';
postgres=# ALTER ROLE myprojectuser SET timezone TO 'UTC';
```

Now, we can give our new user access to administer our new database:

```
postgres=# GRANT ALL PRIVILEGES ON DATABASE myproject TO myprojectuser;
```

When you are finished, exit out of the PostgreSQL prompt by typing:

```
postgres=# \q
```

Postgres is now set up so that Django can connect to and manage its database.

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

To do this, we first need access to the `virtualenv` command. We can install this with `pip`.

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

If you are using **Python 2**, upgrade `pip` and install the package by typing:

```
$ sudo -H pip install --upgrade pip
$ sudo -H pip install virtualenv
```

With `virtualenv` installed, we can start forming our project. Create and move into a directory where we can keep our project files:

```
$ mkdir ~/myprojectdir
$ cd ~/myprojectdir
```

Within the project directory, create a Python virtual environment by typing:

```
$ virtualenv myprojectenv
```

This will create a directory called `myprojectenv` within your `myprojectdir` directory. Inside, it will install a local version of Python and a local version of `pip`. We can use this to install and configure an isolated Python environment for our project.

Before we install our project's Python requirements, we need to activate the virtual environment. You can do that by typing:

```
$ source myprojectenv/bin/activate
```

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

Note: When the virtual environment is activated (when your prompt has `(myprojectenv)` preceding it), use `pip` instead of `pip3`, even if you are using Python 3. The virtual

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

Creating and Configuring a New Django Project

With our Python components installed, we can create the actual Django project files.

Creating the Django Project

Since we already have a project directory, we will tell Django to install the files here. It will create a second level directory with the actual code, which is normal, and place a management script in this directory. The key to this is that we are defining the directory explicitly instead of allowing Django to make decisions relative to our current directory:

```
(myprojectenv) $ django-admin startproject myproject ~/myprojectdir
```

At this point, your project directory (`~/myprojectdir` in our case) should have the following content:

- `~/myprojectdir/manage.py`: A Django project management script.
- `~/myprojectdir/myproject/`: The Django project package. This should contain the `__init__.py`, `settings.py`, `urls.py`, `asgi.py`, and `wsgi.py` files.
- `~/myprojectdir/myprojectenv/`: The virtual environment directory we created earlier.

Adjusting the Project Settings

The first thing we should do with our newly created project files is adjust the settings.

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

incoming requests with a **Host** header that is not in this list will raise an exception. Django requires that you set this to prevent a certain class of security vulnerability.

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

Note: Be sure to include `localhost` as one of the options since we will be proxying connections through a local Nginx instance.

~/myprojectdir/myproject/settings.py

```
...  
# The simplest case: just add the domain name(s) and IP addresses of your Django server  
# ALLOWED_HOSTS = [ 'example.com', '203.0.113.5']  
# To respond to 'example.com' and any subdomains, start the domain with a dot  
# ALLOWED_HOSTS = [ '.example.com', '203.0.113.5']  
ALLOWED_HOSTS = [ 'your_server_domain_or_IP', 'second_domain_or_IP', ..., 'localhost']
```

Next, find the section that configures database access. It will start with `DATABASES`. The configuration in the file is for a SQLite database. We already created a PostgreSQL database for our project, so we need to adjust the settings.

Change the settings with your PostgreSQL database information. We tell Django to use the `psycopg2` adaptor we installed with `pip`. We need to give the database name, the database username, the database user's password, and then specify that the database is located on the local computer. You can leave the `PORT` setting as an empty string:

~/myprojectdir/myproject/settings.py

...

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

```
}  
}
```

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

~/myprojectdir/myproject/settings.py

```
...  
  
STATIC_URL = '/static/'  
import os  
STATIC_ROOT = os.path.join(BASE_DIR, 'static/')
```

Save and close the file when you are finished.

Completing Initial Project Setup

Now, we can migrate the initial database schema to our PostgreSQL database using the management script:

```
(myprojectenv) $ ~/myprojectdir/manage.py makemigrations  
(myprojectenv) $ ~/myprojectdir/manage.py migrate
```

Create an administrative user for the project by typing:

```
(myprojectenv) $ ~/myprojectdir/manage.py createsuperuser
```

You will have to select a username, provide an email address, and choose and confirm a password.

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

You will have to confirm the operation. The static files will then be placed in a directory called `static` within your project directory.

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

```
(myprojectenv) $ sudo ufw allow 8000
```

Finally, you can test our your project by starting up the Django development server with this command:

```
(myprojectenv) $ ~/myprojectdir/manage.py runserver 0.0.0.0:8000
```

In your web browser, visit your server's domain name or IP address followed by `:8000`:

```
http://server_domain_or_IP:8000
```

You should receive the default Django index page:

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

django

[View release notes](#) for Django 3.0**Sign up for our newsletter** Get the latest tutorials on SysAdmin and open source topics. ✕**Sign Up**

You are seeing this page because `DEBUG=True` is in your settings file and you have not configured any URLs.

**Django Documentation**
Topics, references, & how-to's**Tutorial: A Polling App**
Get started with Django**Django Community**
Connect, get help, or contribute

If you append `/admin` to the end of the URL in the address bar, you will be prompted for the administrative username and password you created with the `createsuperuser` command:

Django administration

Username:

Password:

Log in

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

Django administration

WELCOME, SAMMY. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)**Sign up for our newsletter** Get the latest tutorials on SysAdmin and open source topics. ✕**Sign Up**

When you are finished exploring, hit **CTRL-C** in the terminal window to shut down the development server.

Testing Gunicorn's Ability to Serve the Project

The last thing we want to do before leaving our virtual environment is test Gunicorn to make sure that it can serve the application. We can do this by entering our project directory and using `gunicorn` to load the project's WSGI module:

```
(myprojectenv) $ cd ~/myprojectdir
(myprojectenv) $ gunicorn --bind 0.0.0.0:8000 myproject.wsgi
```

This will start Gunicorn on the same interface that the Django development server was running on. You can go back and test the app again.

Note: The admin interface will not have any of the styling applied since Gunicorn does not know how to find the static CSS content responsible for this.

We passed Gunicorn a module by specifying the relative directory path to Django's `wsgi.py` file, which is the entry point to our application, using Python's module syntax. Inside of this file, a function called `application` is defined, which is used to communicate with the application. To learn more about the WSGI specification, click [here](#).

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

The virtual environment indicator in your prompt will be removed.

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

The Gunicorn socket will be created at boot and will listen for connections. When a connection occurs, systemd will automatically start the Gunicorn process to handle the connection.

Start by creating and opening a systemd socket file for Gunicorn with `sudo` privileges:

```
$ sudo nano /etc/systemd/system/gunicorn.socket
```

Inside, we will create a `[Unit]` section to describe the socket, a `[Socket]` section to define the socket location, and an `[Install]` section to make sure the socket is created at the right time:

/etc/systemd/system/gunicorn.socket

```
[Unit]
Description=gunicorn socket

[Socket]
ListenStream=/run/gunicorn.sock

[Install]
WantedBy=sockets.target
```

Save and close the file when you are finished.

Next, create and open a systemd service file for Gunicorn with `sudo` privileges in your

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

Start with the `[Unit]` section, which is used to specify metadata and dependencies. We'll put a description of our service here and tell the init system to only start this after

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

```
Requires=gunicorn.socket
After=network.target
```

Next, we'll open up the `[Service]` section. We'll specify the user and group that we want to process to run under. We will give our regular user account ownership of the process since it owns all of the relevant files. We'll give group ownership to the `www-data` group so that Nginx can communicate easily with Gunicorn.

We'll then map out the working directory and specify the command to use to start the service. In this case, we'll have to specify the full path to the Gunicorn executable, which is installed within our virtual environment. We will bind the process to the Unix socket we created within the `/run` directory so that the process can communicate with Nginx. We log all data to standard output so that the `journald` process can collect the Gunicorn logs. We can also specify any optional Gunicorn tweaks here. For example, we specified 3 worker processes in this case:

`/etc/systemd/system/gunicorn.service`

```
[Unit]
Description=gunicorn daemon
Requires=gunicorn.socket
After=network.target

[Service]
User=sammy
Group=www-data
WorkingDirectory=/home/sammy/myprojectdir
```

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

Finally, we'll add an `[Install]` section. This will tell systemd what to link this service to if we enable it to start at boot. We want this service to start when the regular multi-user

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

```
After=network.target
```

```
[Service]
```

```
User=sammy
```

```
Group=www-data
```

```
WorkingDirectory=/home/sammy/myprojectdir
```

```
ExecStart=/home/sammy/myprojectdir/myprojectenv/bin/gunicorn \
    --access-logfile - \
    --workers 3 \
    --bind unix:/run/gunicorn.sock \
    myproject.wsgi:application
```

```
[Install]
```

```
WantedBy=multi-user.target
```

With that, our systemd service file is complete. Save and close it now.

We can now start and enable the Gunicorn socket. This will create the socket file at `/run/gunicorn.sock` now and at boot. When a connection is made to that socket, systemd will automatically start the `gunicorn.service` to handle it:

```
$ sudo systemctl start gunicorn.socket
$ sudo systemctl enable gunicorn.socket
```

We can confirm that the operation was successful by checking for the socket file.

Checking for the Gunicorn Socket File

Check the status of the process to find out whether it was able to start:

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

Output

- gunicorn.socket - gunicorn socket

Loaded: loaded (/etc/systemd/system/gunicorn.socket; enabled; vendor preset: enabled)

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

Next, check for the existence of the `gunicorn.sock` file within the `/run` directory:

```
$ file /run/gunicorn.sock
```

Output

```
/run/gunicorn.sock: socket
```

If the `systemctl status` command indicated that an error occurred or if you do not find the `gunicorn.sock` file in the directory, it's an indication that the Gunicorn socket was not able to be created correctly. Check the Gunicorn socket's logs by typing:

```
$ sudo journalctl -u gunicorn.socket
```

Take another look at your `/etc/systemd/system/gunicorn.socket` file to fix any problems before continuing.

Testing Socket Activation

Currently, if you've only started the `gunicorn.socket` unit, the `gunicorn.service` will not be active yet since the socket has not yet received any connections. You can check this by typing:

```
$ sudo systemctl status gunicorn
```

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

To test the socket activation mechanism, we can send a connection to the socket through `curl` by typing:

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

```
$ sudo systemctl status gunicorn
```

Output

```
● gunicorn.service - gunicorn daemon
   Loaded: loaded (/etc/systemd/system/gunicorn.service; disabled; vendor preset: enabled)
   Active: active (running) since Fri 2020-06-26 18:52:21 UTC; 2s ago
 TriggeredBy: ● gunicorn.socket
   Main PID: 22914 (gunicorn)
     Tasks: 4 (limit: 1137)

   Memory: 89.1M
   CGroup: /system.slice/gunicorn.service
           └─22914 /home/sammy/myprojectdir/myprojectenv/bin/python /home/sammy/myproject
           └─22927 /home/sammy/myprojectdir/myprojectenv/bin/python /home/sammy/myproject
           └─22928 /home/sammy/myprojectdir/myprojectenv/bin/python /home/sammy/myproject
           └─22929 /home/sammy/myprojectdir/myprojectenv/bin/python /home/sammy/myproject

Jun 26 18:52:21 django-tutorial systemd[1]: Started gunicorn daemon.
Jun 26 18:52:21 django-tutorial gunicorn[22914]: [2020-06-26 18:52:21 +0000] [22914] [INFO]
Jun 26 18:52:21 django-tutorial gunicorn[22914]: [2020-06-26 18:52:21 +0000] [22914] [INFO]
Jun 26 18:52:21 django-tutorial gunicorn[22914]: [2020-06-26 18:52:21 +0000] [22914] [INFO]
Jun 26 18:52:21 django-tutorial gunicorn[22927]: [2020-06-26 18:52:21 +0000] [22927] [INFO]
Jun 26 18:52:21 django-tutorial gunicorn[22928]: [2020-06-26 18:52:21 +0000] [22928] [INFO]
Jun 26 18:52:21 django-tutorial gunicorn[22929]: [2020-06-26 18:52:21 +0000] [22929] [INFO]
```

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

```
$ sudo journalctl -u gunicorn
```

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

Make sure you troubleshoot the above issues before continuing.

Configure Nginx to Proxy Pass to Gunicorn

Now that Gunicorn is set up, we need to configure Nginx to pass traffic to the process.

Start by creating and opening a new server block in Nginx's `sites-available` directory:

```
$ sudo nano /etc/nginx/sites-available/myproject
```

Inside, open up a new server block. We will start by specifying that this block should listen on the normal port 80 and that it should respond to our server's domain name or IP address:

`/etc/nginx/sites-available/myproject`

```
server {  
    listen 80;  
    server_name server_domain_or_IP;  
}
```

Next, we will tell Nginx to ignore any problems with finding a favicon. We will also tell it where to find the static assets that we collected in our `~/myprojectdir/static` directory. All of these files have a standard URI prefix of `"/static"`, so we can create a location block

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

```
location = /favicon.ico { access_log off; log_not_found off; }
location /static/ {
    root /home/sammy/myprojectdir;
```

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

/etc/nginx/sites-available/myproject

```
server {
    listen 80;
    server_name server_domain_or_IP;

    location = /favicon.ico { access_log off; log_not_found off; }
    location /static/ {
        root /home/sammy/myprojectdir;
    }

    location / {
        include proxy_params;
        proxy_pass http://unix:/run/gunicorn.sock;
    }
}
```

Save and close the file when you are finished. Now, we can enable the file by linking it to the `sites-enabled` directory:

```
$ sudo ln -s /etc/nginx/sites-available/myproject /etc/nginx/sites-enabled
```

Test your Nginx configuration for syntax errors by typing:

```
$ sudo nginx -t
```

If no errors are reported, proceed and restart Nginx by typing:

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

well:

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

Note: After configuring Nginx, the next step should be securing traffic to the server using SSL/TLS. This is important because without it, all information, including passwords are sent over the network in plain text.

If you have a domain name, the easiest way to get an SSL certificate to secure your traffic is using Let's Encrypt. Follow [this guide](#) to set up Let's Encrypt with Nginx on Ubuntu 20.04. Follow the procedure using the Nginx server block we created in this guide.

Troubleshooting Nginx and Gunicorn

If this last step does not show your application, you will need to troubleshoot your installation.

Nginx Is Showing the Default Page Instead of the Django Application

If Nginx displays the default page instead of proxying to your application, it usually means that you need to adjust the `server_name` within the `/etc/nginx/sites-available/myproject` file to point to your server's IP address or domain name.

Nginx uses the `server_name` to determine which server block to use to respond to requests. If you receive the default Nginx page, it is a sign that Nginx wasn't able to match the request to a sever block explicitly, so it's falling back on the default block defined in `/etc/nginx/sites-available/default`.

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

A 502 error indicates that Nginx is unable to successfully proxy the request. A wide range of configuration problems express themselves with a 502 error, so more information is

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

```
$ sudo tail -F /var/log/nginx/error.log
```

Now, make another request in your browser to generate a fresh error (try refreshing the page). You should receive a fresh error message written to the log. If you look at the message, it should help you narrow down the problem.

You might receive the following message:

connect() to unix:/run/gunicorn.sock failed (2: No such file or directory)

This indicates that Nginx was unable to find the `gunicorn.sock` file at the given location. You should compare the `proxy_pass` location defined within `/etc/nginx/sites-available/myproject` file to the actual location of the `gunicorn.sock` file generated by the `gunicorn.socket` systemd unit.

If you cannot find a `gunicorn.sock` file within the `/run` directory, it generally means that the systemd socket file was unable to create it. Go back to the [section on checking for the Gunicorn socket file](#) to step through the troubleshooting steps for Gunicorn.

connect() to unix:/run/gunicorn.sock failed (13: Permission denied)

This indicates that Nginx was unable to connect to the Gunicorn socket because of permissions problems. This can happen when the procedure is followed using the root user instead of a `sudo` user. While systemd is able to create the Gunicorn socket file, Nginx is unable to access it.

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

```
$ namei -l /run/gunicorn.sock
```

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

The output displays the permissions of each of the directory components. By looking at the permissions (first column), owner (second column) and group owner (third column), we can figure out what type of access is allowed to the socket file.

In the above example, the socket file and each of the directories leading up to the socket file have world read and execute permissions (the permissions column for the directories end with `r-x` instead of `---`). The Nginx process should be able to access the socket successfully.

If any of the directories leading up to the socket do not have world read and execute permission, Nginx will not be able to access the socket without allowing world read and execute permissions or making sure group ownership is given to a group that Nginx is a part of.

Django Is Displaying: “could not connect to server: Connection refused”

One message that you may receive from Django when attempting to access parts of the application in the web browser is:

```
OperationalError at /admin/login/
could not connect to server: Connection refused
    Is the server running on host "localhost" (127.0.0.1) and accepting
    TCP/IP connections on port 5432?
```

This indicates that Django is unable to connect to the Postgres database. Make sure that

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

```
$ sudo systemctl start postgresql
$ sudo systemctl enable postgresql
```

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

of them in turn and look for messages indicating problem areas.

The following logs may be helpful:

- Check the Nginx process logs by typing: `sudo journalctl -u nginx`
- Check the Nginx access logs by typing: `sudo less /var/log/nginx/access.log`
- Check the Nginx error logs by typing: `sudo less /var/log/nginx/error.log`
- Check the Gunicorn application logs by typing: `sudo journalctl -u gunicorn`
- Check the Gunicorn socket logs by typing: `sudo journalctl -u gunicorn.socket`

As you update your configuration or application, you will likely need to restart the processes to adjust to your changes.

If you update your Django application, you can restart the Gunicorn process to pick up the changes by typing:

```
$ sudo systemctl restart gunicorn
```

If you change Gunicorn socket or service files, reload the daemon and restart the process by typing:

```
$ sudo systemctl daemon-reload
```

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

These commands are helpful for picking up changes as you adjust your configuration.

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

Django makes creating projects and applications simple by providing many of the common pieces, allowing you to focus on the unique elements. By leveraging the general tool chain described in this article, you can easily serve the applications you create from a single server.

Was this helpful?

Yes

No



[Report an issue](#)

About the authors



Erin Glass

Open source advocate and lover of education, culture, and community.

Still Looking for an answer?

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

RELATED

Sign up for our newsletter

Get the latest tutorials on SysAdmin and open source topics.

✕

Sign Up

Get courses & tools that help you grow as a developer or small business owner

Join Now

How To Use Transactions in MongoDB

 [Tutorial](#)

How To Use an SQLite Database in a Flask Application

 [Tutorial](#)

Comments

47 Comments

B

I















Leave a comment...

Sign In to Comment

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

 [eglass](#)  July 29, 2020
1 Thanks, juangiordana!

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. 

Enter your email address

Sign Up

 [eglass](#)  July 29, 2020
0 Thanks, Somac! The one-click is pre-configured with Django along with Nginx, Certbot, Gunicorn, Postgres, and Postfix to get you up and running more quickly. You can follow this tutorial to do more customizations or to walk through the steps of setting up your own infrastructure.

[Reply](#) [Report](#)

 [alcatrazz627](#) July 27, 2020
2 I have referred to this article(this and the previous versions) countless times. Thank you so much!

[Reply](#) [Report](#)

 [eglass](#)  July 29, 2020
0 Thanks, alcatrazz627!

[Reply](#) [Report](#)

 [lexokechukwu](#) July 31, 2020
0 Thank you for the tutorial. I successfully completed this and got the 'Django install worked successfully!' page on my droplets IP. However I tried pasting my project files unto the root SFTP using MobaXterm but the page has remained the same. installed apps, urls and templates have been updated but no results.

[Reply](#) [Report](#)

 [Ezigboiti](#) August 5, 2020
0 [@lexokechukwu](#), try checking your logs. I'm a beginner developer, and I had all sorts of problems when I deployed my first app. It took me about 2days(working on it part time)

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

Also thank you so very much (“,”)

[Reply](#) [Report](#)

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

1 Yes, I had the same problem. When you are updating the django system configuration file (~/myprojectdir/myproject/settings.py) with the path to the static content (`STATICROOT = os.path.join(BASEDIR, 'static/')`), it doesn't know about os. At the top of the file you have to add “import os” (without the quotes). This imports the python os library into the settings.py program so that it can be used to join the BASE_DIR and the 'static/' directory.

[Reply](#) [Report](#)

^ loudeye September 18, 2020

0 import os is at the top of the settings.py file and it still does not work. I am lost.

[Reply](#) [Report](#)

^ adew September 21, 2020

0 So it should look like this at the top

```
"""
Django settings for IonizeAI project.

Generated by 'django-admin startproject' using Django 3.1.1.

For more information on this file, see
https://docs.djangoproject.com/en/3.1/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/3.1/ref/settings/
"""
```

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

```
# Static files (CSS, JavaScript, Images)
```

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

[Reply](#) [Report](#)

 [thbrianwong](#) October 6, 2020

1 os is not the default method after Django 3.1
It is recommend to use pathlib for clarity

Replace

```
STATIC_ROOT = os.path.join(BASE_DIR, 'static/')
```

With

```
STATIC_ROOT = BASE_DIR / 'static'
```

Also I would recommend the use of sqlite in lieu of postgresql for ease of use.

[Reply](#) [Report](#)

 [Walidovski](#) October 3, 2020

2 Perfect tutorial, managed to host multiple django apps using the same method just made separate config files for each app:

```
/etc/nginx/sites-available/project1
```

```
/etc/nginx/sites-available/project2
```

```
project1:
```

```
    unicorn.socket
```

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

[Reply](#) [Report](#) [fkrasniq](#) January 24, 2021**Sign up for our newsletter** Get the latest tutorials on SysAdmin and open source topics. **Sign Up**

not working,

already trying something like this:

```
server {
    server_name example.com www.example.com;
    location /static {
        autoindex on;
        alias /home/myusername/myproject/static/;
    }
    location /media {
        autoindex on;
        alias /home/myusername/myproject/media/;
    }
    location / {
        proxy_pass http://127.0.0.1:8000;
    }
}
```

but still not working for production mode.

[Reply](#) [Report](#) [calfano1999](#) January 25, 2021

0 Did you ever find a solution to this? Currently having the same issue

[Reply](#) [Report](#) [profmorten](#) April 10, 2021

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

After trying different combinations what seemed to work for me (something I tried before too) was this:

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. 

Sign Up

```
root /home/user/site;  
}
```

```
location /media/ {  
    root /home/user/site;  
}
```

```
sudo systemctl restart gunicorn  
sudo systemctl restart nginx
```

I hope this is the setting that solves it. I will wait a few days and confirm.

[Reply](#) [Report](#)

 [niceguy3088](#) May 29, 2021
 this worked. for the static files issue.

Nginx conf in /sites-available/

```
location /static/ {  
    root /your/project/sitedirectory;  
}
```

Debug does not need to be True.

[Reply](#) [Report](#)

 [dungdh286](#) November 25, 2020
 if i want deploy 2 web server how to set up this?Thank for great introduction

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

- on localhost the default nginx page is displayed (even though I removed default from sites-available and sites-enabled)

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

Does anybody has an idea what's wrong with my configuration?

[Reply](#) [Report](#)

 **fkrasniq** January 24, 2021

 Assuming everything else is configured correctly, please read all the log files for each platform (gunicorn, nginx, django) they will steer you in the right direction:

make sure you restart gunicorn after making changes to your ALLOWED_HOSTS= setting on django. the line I believe is:
-sudo systemctl restart gunicorn

If that does not work I would also restart nginx and the sockets, good luck!

[Reply](#) [Report](#)

 **ralu** January 26, 2021

 Thank you!

[Reply](#) [Report](#)

 **sqikaapp** February 13, 2021

 How do resolve this?

Feb 13 07:35:15 sqika systemd[1]: Started gunicorn daemon.

Feb 13 07:35:15 sqika systemd[68549]: gunicorn.service: Failed to execute command:
Permission denied<^>

[Reply](#) [Report](#)

 **samuelishimwe** February 13, 2021

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

^ [rachellevhu123](#) March 11, 2021

0 This is the best, most clear article I've followed. Thank you so much!

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. X

Enter your email address

Sign Up

also no errors present in any logs also indicating that the configuration is correct.
All my checks also indicate the same.

I do restart Gunicorn + Nginx whenever I make any changes to settings.py...etc.

I am bewildered to the point where I would just prefer a stable reliable old school Apache server over having to spend so much time on serving static files. This has been the most time-consuming part of Heroku - Digitalocean - Django. They make the hard things easy, then again they make the simple things difficult.

[Reply](#) [Report](#)

^ [crestsource](#) April 26, 2021

0 I have used same tutorial but nginx shows 403 for all media files, could anyone tell me any solution on this.

[Reply](#) [Report](#)

^ [khojasahil](#) October 21, 2021

0 Hi I'm getting the same error. Did you solve it ?

[Reply](#) [Report](#)

^ [devildc](#) October 22, 2021

0 hey, have you solved it?

[Reply](#) [Report](#)

^ [ruskiycoder](#) May 20, 2021

0 In the further troubleshooting section, it explains how to access all the logs. How do I tail those logs live so it doesn't exit out of the log in the terminal. I tried:

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

Nothing keeps the log going without exiting out of it. Also whatever the command is to keep the log going, would it work for all the other logs as well?

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

website running)

[Reply](#) [Report](#)

 **idenesten** June 26, 2021

0 God bless you, Erin!

After 3 days of pain and suffering your article is like a divine intervention =)

[Reply](#) [Report](#)

Load More Comments



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.



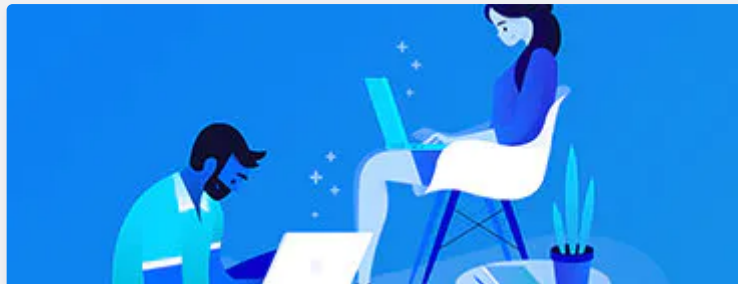
We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

Sign up for our newsletter Get the latest tutorials on SysAdmin and open source topics. ✕

Sign Up

Working on improving health and education, reducing inequality, and spurring economic growth? We'd like to help.



BECOME A CONTRIBUTOR

You get paid; we donate to tech nonprofits.

Featured on [Community](#) [Kubernetes Course](#) [Learn Python 3](#) [Machine Learning in Python](#)
[Getting started with Go](#) [Intro to Kubernetes](#)

DigitalOcean Products [Virtual Machines](#) [Managed Databases](#) [Managed Kubernetes](#) [Block Storage](#)
[Object Storage](#) [Marketplace](#) [VPC](#) [Load Balancers](#)

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand

[Learn More](#)

Sign up for our newsletter

Get the latest tutorials on SysAdmin and open source topics.

✕

Sign Up



© 2021 DigitalOcean, LLC. All rights reserved.

Company

- About
- Leadership
- Blog
- Careers
- Partners
- Referral Program
- Press
- Legal
- Security & Trust Center

Products

- Pricing
- Products Overview
- Droplets
- Kubernetes
- Managed Databases
- Spaces

Community

- Tutorials
- Q&A
- Tools and Integrations
- Tags
- Write for DigitalOcean
- Presentation Grants

Contact

- Get Support
- Trouble Signing In?
- Sales
- Report Abuse
- System Status
- Share your ideas

We use cookies to provide our services and for analytics and marketing. To find out more about our use of cookies, please see our [Privacy Policy](#) and [Cookie and Tracking Notice](#). By continuing to browse our website, you agree to our use of cookies.

I understand